

Towards Autocomplete Strategies for Visualization Construction

Wei Wei *
University of Victoria
University of Calgary
Télécom Paris

Samuel Huron†
Télécom Paris, CNRS I3, Institut Polytechnique de Paris

Yvonne Jansen‡
Univ. Bordeaux, CNRS, Inria, LaBRI

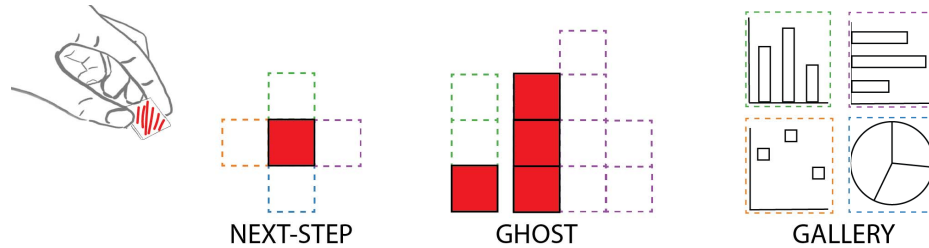


Figure 1: The three *visualization autocomplete* strategies identified in our study. The dashed boxes in different colors represent autocomplete suggestions. Based on the token(s) that has been placed, NEXT-STEP provides suggestions for a single next visual mapping operation. GHOST provides situated partial or completed visualization recommendations. GALLERY provides a gallery of completed visual mapping options.

ABSTRACT

Constructive visualization uses physical data units - tokens - to enable non-experts to create personalized visualizations engagingly. However, its physical nature limits efficiency and scalability. One potential solution to address this issue is autocomplete. By providing automated suggestions while still allowing for manual intervention, autocomplete can expedite visualization construction while maintaining expressivity. We conduct a speculative design study to examine how people would like to interact with a visualization authoring system that supports autocomplete. Our study identifies three types of autocomplete strategies and gains insights for designing future visualization authoring tools with autocomplete functionality. A free copy of this paper and all supplemental materials are available on our online repository: https://osf.io/nu4z3/?view_only=594baee54d114a99ab381886fb32a126.

Keywords: Autocomplete, constructive visualization, visualization authoring, physicalization, automation, expressivity, design.

Index Terms: Human-centered computing—Visualization

1 INTRODUCTION

Constructive visualization is a visualization authoring paradigm that emphasizes the act of manually assembling tokens mapped to data [8, 9, 32]. Its accessible method enables non-experts to build novel visualizations in an engaging way.

While the process of manual assembly facilitates expressivity (the support for diverse visualization types), its ability to scale to thousands or millions of data points is limited. In the field of visualization, there has been a long-standing interest in using automation to assist in creating scalable visualizations in an efficient manner [17, 18, 25, 29]. In this paper, we are interested in how to apply automation mechanisms to constructive visualization while maintaining expressivity. To achieve this, we explore autocomplete

as a way to assist visualization construction that supports both automation and manual assembly. Autocomplete [4] is a widespread mechanism that aids in quickly obtaining desired outcomes based on user input. It supports manual operation while providing automated suggestions which can be accepted to speed up interaction or ignored to continue manually. Although originally used for text entry, autocomplete is now employed in various fields ([1, 6, 12, 16]). Inspired by these works, we define *visualization autocomplete* as **a function or strategy that supplies one or more options of predicted partial or complete visualization**, which could include any step of the visualization system of the infovis pipeline [10]. We conducted a speculative design study to explore how autocomplete could be designed in the context of constructive visualization.

One could imagine different ways to technically enable autocomplete mechanisms for constructive visualization (e.g., digital tokens, swarm bots [15] or XR technologies). However, explicitly considering all these technologies constrains the potential interaction design and might introduce some level of indirection [5]. Therefore, we focus here on exploring interaction design rather than the technology enabling potential interactions. Consequently, we asked 15 participants to sketch how they might want to interact with a *visualization autocomplete* system without considering any technical constraints.

Participants generated 93 pages of sketches which we analyzed through a thematic analysis. We identified three types of autocomplete strategies (shown in Figure 1) that conclude how a system could provide visual mapping recommendations and automation of the visualization rendering during the construction process. We then illustrate each strategy in how they help people explore the visual mapping decision tree. We further discuss the tradeoff between automation and expressivity embodied in the three strategies and potential schemes to facilitate better visualization construction. Our work is a first attempt to explore how a system can support visual mapping autocomplete while enabling direct manipulation of the visual variables. We contribute by (1) proposing the first definition of *visualization autocomplete*, (2) identifying three interaction design strategies, and (3) discussing future research and design possibilities.

2 RELATED WORK

2.1 Autocomplete

Autocomplete systems were initially designed to speed up text entry by providing suggestions for completing users' input in specialized

*e-mail: weiwei@uvic.ca

†e-mail: samuel.huron@telecom-paris.fr

‡e-mail: yvonne.jansen@cnrs.fr

input fields such as Unix command-line fields [14], file location fields [22] and email address fields [22]. Their applications then expanded to include word and phrase completion in general text editors such as Microsoft Word. Autocomplete also goes beyond text entry and is applied in various fields of information science to improve efficiency (e.g., programming assistance [1, 12], breadboard design [16], and enhancing gestural interaction [6]). In general, autocomplete is “a widely used mechanism to get to a desired piece of information quickly and with as little knowledge and effort as possible [4]”.

In the field of information visualization, a limited number of studies have explored the use of autocomplete for visual analysis as well as building visualization. Aiming to help the data discovery process, Setlur et al. [27] developed a system that incorporated text- and widget-based autocompletion to support query formulation in natural language interfaces for visual analysis. Another example, VisComplete [13], provides assisted construction of visualization pipelines. When a user is adding a module to the pipeline, VisComplete suggests a completion.

2.2 Constructive Visualization

There is a continuum between two ends that describe the approaches employed by visualization authoring tools [9, 19]: **top-down** and **bottom-up**. When building a visualization, top-down approaches start by defining high-level abstractions (e.g., visualization types, dimensions, and data mapping) and then populate them with data. Examples include the ‘Recommended Charts’ in Microsoft Excel and the visualization browser in Voyager [31]. In contrast, bottom-up approaches start with individual data points and then build progressively towards defining higher-level structures, such as axes. A vital concept related to bottom-up approaches is constructive visualization - “the act of constructing a visualization by assembling blocks, that have previously been assigned a data unit through a mapping” [8].

One digital implementation that employs the paradigm of constructive visualization is iVoLVER [21], a web-based constructive visualization authoring tool that supports the manipulation of atomic elements of data. The authors of iVoLVER conducted a comparison study between iVoLVER (with bottom-up approaches) and Tableau (with top-down approaches) [19], and an investigation of how to reconcile automation with the benefits of constructive visualization [20].

Our work also aims to facilitate constructive visualization through automation while maintaining expressivity. However, instead of implementing and studying a specific system, we study how people might want to interact with an ideal system. This approach is inspired by studying the interaction, not the interface [5]. To focus on interaction at the level of the visual mark [23, 24], we used similar tangible tokens as previous constructive visualization studies and activities [9, 30, 32]. Furthermore, the simplicity of tokens allows for early exploration of various design alternatives, making them a powerful tool for our preliminary exploration [28], and it is probably one of the earliest information processing tools human have used [26].

3 THE STUDY

We aim to understand how people might want to interact with a system able to autocomplete their visualization. To study the desired interactions without influencing our participants towards specific interfaces or technology, we decided to run a study in the form of a speculative design workshop [3]. The study consisted of four design workshops. Each workshop was run with a group of 3-5 participants.

3.1 Participants

We recruited 15 participants from a local university (8 females and 7 males, 7 aged 18-24 and 8 aged 25-44). We requested participants to complete a pre-study demographic survey, where we gathered information regarding their proficiency and frequency of metrics

associated with visualization, interaction design, and XR technology. We inquired about XR technology to investigate the potential impact of participants’ XR experience on their sketches. The demographic data are available in our online repository. Each workshop consisted of 3 sessions: **Construction**, **Ideation**, and **Discussion**. The participants carried out the first two alone and were involved in a group discussion in the last one. Each workshop took about 1.5 hours.

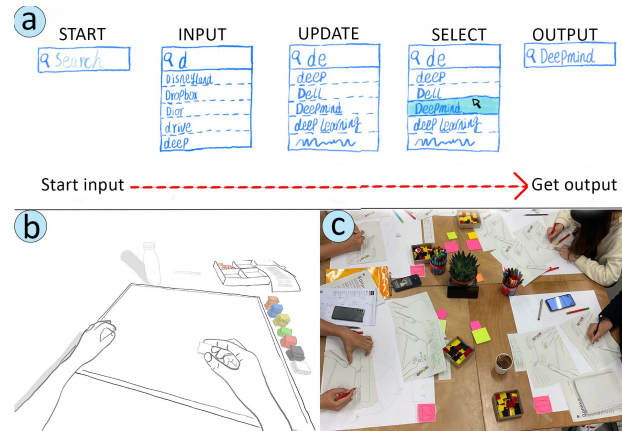


Figure 2: (a) The diagram illustrates the mechanism of the search engine autocomplete feature. (b) The sketch template. (c) Participants were sketching their ideas.

3.1.1 Construction

The construction session introduced participants to the concept of constructive visualization. The setup followed Huron et al. [9]’s study (including a box of tokens, a printed dataset, and an A2 paper canvas). We simplified the data to positive integers up to eight to eliminate the need for explicit token mapping, which gave participants more time for manual assembling instead of mathematical calculations. At the workshop’s outset, participants were informed of a person seeking their advice on expenses. Their objective was to assist in visualizing the monthly budget using tokens. We asked participants to create as many visualization alternatives as possible through tokens in 15 minutes. In case participants got stuck creating visualizations, we prepared four low-level analysis tasks based on the work of Amar et al. [2]: *Filter*, *Compute Derived Value*, *Sort*, and *Correlate*. We chose these four out of the ten in Amar et al.’s work because we wanted to motivate participants to physically arrange and update their visualization, rather than only mentally thinking.

3.1.2 Ideation

We introduced the ideation session by presenting the concept of autocomplete as it is used in search engines through a sketch (See Figure 2a). When people use a search engine, they open a page, input some letters to form a word, and the search engine shows some recommended completion until the user selects the desired one and submits it. We illustrated the five main steps of autocomplete systems: *Start*, *input*, *update*, *select* and *output*. We presented the illustration to participants and encouraged them to apply this metaphor to the construction process of visualizations.

We then asked participants to imagine how, in the near future, a fictional smart system with no technological limitation could help them to create visualizations following the five steps of autocomplete. We asked them to present their ideas in the form of pen and paper sketches. To guide their ideation process, we 1) provided paper with a template of a scene in which someone is placing the first token on a canvas (see Figure 2b), and 2) displayed a list of questions on a screen while they were sketching:

1. What would the empty input canvas field look like?
2. When you put the first token, how would the system show the suggestions for alternative visualizations?
3. When you add a new token, how would the system update its suggestions?
4. How would you select the suggestion(s) that you want?
5. How would the system show the final visualization after the selection is done?

Participants had 15 minutes to sketch their answers on the sketch template (Figure 2c).

3.1.3 Discussion

Participants were asked to present and discuss their sketches with each other. They had 2-3 minutes to present their sketches to the group, explaining how a user can interact with their visualization autocomplete authoring system. After the presentation, participants were encouraged to discuss the benefits as well as the limitations of each idea. This session was video-recorded.

3.2 Data Analysis

We collected 93 pages of sketches which formed 17 design ideas. We excluded 3 of these 17 design ideas that were autocomplete for text instead of visualization. To analyze the sketches, we performed a thematic analysis [7]. The authors conducted two rounds of synchronous collaborative coding sessions on the sketches with different coders. Coders were allowed to make changes to their codes between sessions to improve the identification of diverse phenomena. As each round involved a different set of coders, a third session was conducted with all coders to consolidate the codes. During each session, the coders engaged in discussions until arriving at an internal consensus. In the third round, all codes were thoroughly discussed until a consensus was reached among all coders on the definition and instances of codes. The codebook and detailed coding results are available in our online repository.

4 RESULTS

All participants used solid colorful squares to represent the already placed tokens. Correspondingly, participants used unfilled squares with dashed strokes to indicate the tokens yet to be placed. We describe a ‘step’ as the operation of placing a single token on the canvas, which defines a visual mapping for that token. In general, constructing a visualization involves more than one step. Participants produced 14 design ideas (I1-I14). From these, we identified

three autocomplete strategies (NEXT-STEP (2/14), GHOST (5/14), GALLERY (3/14), and the remainder did not specify an autocomplete strategy). Sample sketches can be found in Figure 3.

4.1 NEXT-STEP

NEXT-STEP (sketches with cyan labels in Figure 3) bases its suggestions on proximity information of existing tokens, primarily the last placed token. The basic approach of NEXT-STEP is to highlight available positions around the previous visual mapping operation, such as front, back, left, and right. I5 illustrates this approach, where the system suggests all four positions (highlight in yellow) around the previously placed red token for placing the new token. Furthermore, NEXT-STEP can be more intelligent than simply showing positions in four directions. For instance, I3 ideates a similar idea with more possibilities. There are two more suggested positions at the right and the bottom that align with the previous token vertically and horizontally but are not directly connected to it. Suggestions are accepted by placing the next token, and the NEXT-STEP strategy will update suggestions accordingly for the new next step. This process runs iteratively until a completed visualization forms.

4.2 GHOST

The GHOST (sketches with magenta labels in Figure 3) strategy provides integrated suggestions that autocomplete multiple steps simultaneously. These suggestions are ghosts of a partial or completed visualization. I13 illustrates how the strategy would provide multiple options for partial visualizations based on the token placed. In this idea, after the user places the yellow token, the system generates three options that include a variable number of steps (The participant noted them as option 1, 2, and 3). There is a vertical ghost bar (4 tokens yet to place) and a horizontal ghost bar (5 tokens yet to place). Option 3 is a zigzag visualization that needs 4 more tokens. Users can then place a second token to accept one of these alternatives quickly, and the system will render the tokens automatically.

Moreover, these options can expand to different dimensions. I10 provides ghost autocomplete options for a 2D bar chart and a 3D bar chart. Finally, once enough tokens have been placed, GHOST can provide suggestions for completed visualizations, as depicted in I1 and I9 (I9-1 in Figure 3). The GHOST helps to build a 3D bar chart in I1 and autocompletes a 2D bar chart in I9. I12 shows the same idea with a more powerful system that paints the full picture with only one token placed.

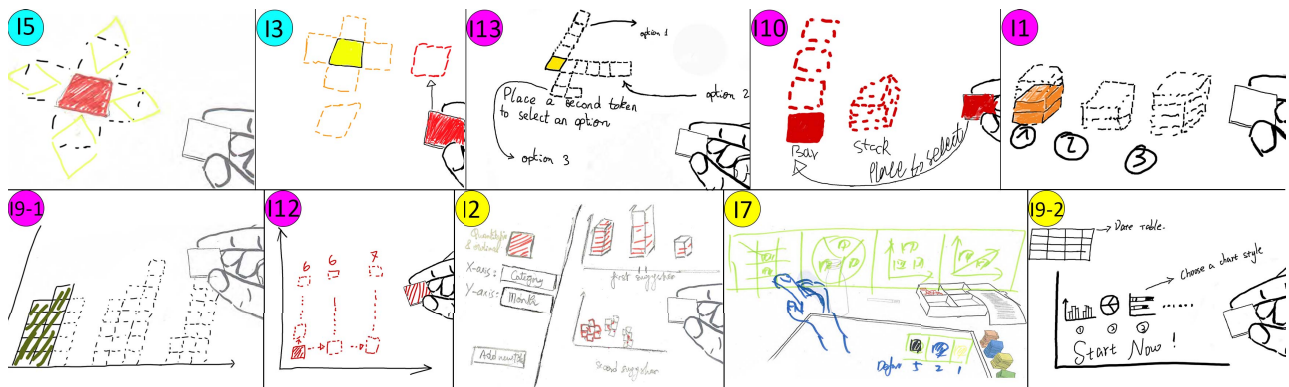


Figure 3: The participants’ design sketches illustrate three autocomplete strategies. I5 and I3 showcase the NEXT-STEP strategy, highlighting available surrounding positions for the next token. The GHOST strategy is more flexible, providing suggestions for 2D (I13, I9-1, I12) and 3D (I10, I1) visualizations, both partial (I13, I10) and full (I1, I9-1, I12). And its suggestions can be based on one token (I13, I10, I12) or multiple tokens (I1, I9-1). Additionally, I2, I7, and I9-2 demonstrate the GALLERY strategy, featuring multiple displays after reading data (I9-2) or defining high-level abstractions (I2, I7).

4.3 GALLERY

The GALLERY (sketches with yellow labels in Figure 3) strategy offers integrated suggestions that autocomplete all the steps of building a visualization. The suggestions are from the perspective of the whole visualization rather than one single construction operation. GALLERY generally provides suggestions through small multiple displays. As a top-down strategy, GALLERY might require users to define high-level abstractions, such as data mapping, before generating multiple visual mapping suggestions. I2 and I7 demonstrate this approach in their sketches. I2 requires users to map data on the left screen, and the system autocompletes visual mapping suggestions on the right screen. I7 presents visual mapping options in AR, using a virtual float screen that appears after users complete data mapping on the table. In both cases, the system automatically renders a visualization once users accept a suggestion.

GALLERY can also be less arbitrary. It can provide suggestions without defining abstractions and allow users to finish the construction manually. For instance, in another sketch of I9 (I9-2 in Figure 3), the system provides a gallery of “chart style” options at the outset, which helps to narrow down the range of possible visualization types without automatically rendering any visualizations. Users must then construct the visualization manually, perhaps with the help of GHOST.

5 DISCUSSION

5.1 Exploring Visual Mapping Decision Tree

The creation of visualizations through manual assembly requires defining visual mappings step by step. This process can be illustrated through a decision tree (Figure 4a). Each node represents a visual mapping or a rendering operation. A path from the root to a leaf portrays a mapping schema that defines a visualization. Each autocomplete strategy helps people to explore the decision tree in a different way.

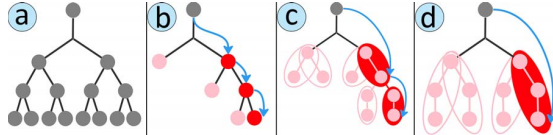


Figure 4: The different ways to explore the visual mapping decision tree. The original decision tree (a) and exploration through NEXT-STEP (b), GHOST(c), and GALLERY (d).

Focusing on sequential steps, one at a time, NEXT-STEP provides a preview for child nodes (nodes in pink, Figure 4b) of the next level in the decision tree. The preview demonstrates the possible visual mappings of the token yet to be placed. With the help of the preview, people are able to make an informed decision by accepting one node (nodes in red) in the preview at every step. By showing the choices step by step, NEXT-STEP helps build a visualization from scratch, especially for non-experts. However, it still requires people to manipulate every token manually, making the authoring process time-consuming.

The GHOST strategy provides suggestions for paths (marked with ellipses in Figure 4c) rather than individual nodes. By presenting suggestions integrating multiple nodes, GHOST allows users to define the visual mapping of multiple tokens simultaneously. As a result, GHOST can speed up the construction process when a desired suggestion is presented. However, identifying which groups of steps of the mapping or the rendering process a GHOST should automatize remains an open question.

Finally, GALLERY is the ultimate version of GHOST, as it provides suggestions for a selection of complete paths from the root to the leaf nodes (Figure 4d). With this high-level of automation, GALLERY autocompletes all visual mapping steps required for a visualization, creating a fast and hands-off experience. However,

while GALLERY is efficient, it also limits the design alternatives that manual assembly can support, and may compromise the expressive nature of the process.

5.2 Automation and Expressivity

The three autocomplete strategies differ in their degree of automation. NEXT-STEP requires manual input for each token. In contrast, GALLERY is highly automated and can exclude human participation in the construction process. GHOST falls in between, incorporating a variety of techniques composed of manual assembly and automation. In exploring design options, NEXT-STEP provides the most expressivity, while GHOST, depending on its design, might reduce expressivity if the suggestions are inappropriate. And GALLERY limits the exploration of choices. This trade-off suggests that increasing automation can decrease expressivity. Achieving the right balance between automation and expressivity is a complex task that requires careful consideration.

One approach is exploiting the potential of GHOST. GHOST is in between NEXT-STEP and GALLERY. This ‘middle’ position makes it the option most likely to facilitate expressive authoring and automated construction. It is also the autocomplete strategy that appeared most frequently in our study (5/14). A perfect GHOST would provide recommendations that automatize the building process and allow people to make all the decisions on the visual mapping. However, identifying what part is building and what part is visual mapping is an open research and design challenge.

Another approach is integrating multiple strategies into a single authoring tool that allows people to switch between them during different phases of visualization creation. For instance, when people want to play with data and explore all possibilities, NEXT-STEP can serve as a ‘non-invasive’ assistant. If people want to reduce the number of manual actions while keeping all or most of the visual mapping choices, GHOST might be more appropriate. If people have a clear idea of a standard visualization, GALLERY might be a valuable tool to expedite the visual mapping and building process. Combining these strategies can provide users with flexibility that balances both automation and expressivity.

5.3 Limitations

Each visualization authoring tool has benefits and constraints influencing the creation process. Sketching on paper does not provide easy undo but allows expressivity and is accessible to the non-expert; coding provides undo and is expressive but requires expertise, while chart editors like Excel allow undoing and are less expressive and require maybe less expertise than coding and more than sketching. For simplicity, this study is limited to autocomplete in the context of a token assembly, which allows undoing and does not require extensive expertise. However, the conception of autocomplete of visualization or physicalizations [10] at the visual mark level is way broader than using tokens or blocks. It could apply to any visualization authoring tool and paradigm. The results of our study are limited to authoring with a specific type of block. Future work could explore what strategy people will suggest with other virtual or physical authoring tools, and processes such as sketching, coding, swarm-bots, or digital fabrication [11], visualization by demonstration [24].

6 CONCLUSION

The concept of visualization autocomplete was defined and introduced in this paper. In a study, we delved into how visualization autocomplete might be by using tangible tokens to construct visualizations. We identify three distinct autocomplete strategies that could aid in automating and simplifying the visual mapping and rendering process during manual visualization authoring. This study provides valuable insights for future visualization authoring tools incorporating autocomplete mechanisms, paving the way for more expressive and automated visualization authoring experiences.

ACKNOWLEDGMENTS

The authors wish to thank Ehud Sharlin and Marcus Friedel. This work was supported by a grant from Mitacs and by the French government funding for the Future program (PIA) grant ANR-21-ESRE-0030 (CONTINUUM).

REFERENCES

- [1] S. Abiteboul, Y. Amsterdamer, T. Milo, and P. Senellart. Auto-completion learning for XML. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*, SIGMOD '12, pp. 669–672. ACM, New York, NY, USA, May 2012. doi: 10.1145/2213836.2213928 1, 2
- [2] R. Amar, J. Eagan, and J. Stasko. Low-level components of analytic activity in information visualization. In *IEEE Symposium on Information Visualization, 2005. INFOVIS 2005.*, pp. 111–117. IEEE, 2005. doi: 10.1109/INFVIS.2005.1532136 2
- [3] J. Auger. Speculative design: crafting the speculation. *Digital Creativity*, 24(1):11–35, 2013. 2
- [4] H. Bast and I. Weber. Type less, find more: fast autocompletion search with a succinct index. In *Proceedings of the 29th annual international ACM SIGIR conference on Research and development in information retrieval*, SIGIR '06, pp. 364–371. ACM, New York, NY, USA, Aug. 2006. doi: 10.1145/1148170.1148234 1, 2
- [5] M. Beaudouin-Lafon. Designing interaction, not interfaces. In *Proceedings of the Working Conference on Advanced Visual Interfaces*, AVI '04, p. 15–22. ACM, New York, NY, USA, 2004. doi: 10.1145/989863.989865 1, 2
- [6] M. Bennett, K. McCarthy, S. O'Modhrain, and B. Smyth. SimpleFlow: Enhancing Gestural Interaction with Gesture Prediction, Abbreviation and Autocompletion. In *Human-Computer Interaction – INTERACT 2011*, Lecture Notes in Computer Science, pp. 591–608. Springer, Berlin, Heidelberg, 2011. doi: 10.1007/978-3-642-23774-4_47 1, 2
- [7] V. Braun and V. Clarke. *Thematic analysis*. American Psychological Association, 2012. 3
- [8] S. Huron, S. Carpendale, A. Thudt, A. Tang, and M. Mauerer. Constructive visualization. In *Proceedings of the 2014 Conference on Designing Interactive Systems*, DIS '14, p. 433–442. ACM, New York, NY, USA, 2014. doi: 10.1145/2598510.2598566 1, 2
- [9] S. Huron, Y. Jansen, and S. Carpendale. Constructing visual representations: Investigating the use of tangible tokens. *IEEE Transactions on Visualization and Computer Graphics*, 20(12):2102–2111, 2014. doi: 10.1109/TVCG.2014.2346292 1, 2
- [10] Y. Jansen and P. Dragicevic. An interaction model for visualizations beyond the desktop. *IEEE Transactions on Visualization and Computer Graphics*, 19(12):2396–2405, 2013. doi: 10.1109/TVCG.2013.134 1, 4
- [11] Y. Jansen, P. Dragicevic, P. Isenberg, J. Alexander, A. Karnik, J. Kildal, S. Subramanian, and K. Hornbæk. Opportunities and challenges for data physicalization. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems*, CHI '15, p. 3227–3236. ACM, New York, NY, USA, 2015. doi: 10.1145/2702123.2702180 4
- [12] N. Khoussainova, Y. Kwon, M. Balazinska, and D. Suciu. SnipSuggest: context-aware autocompletion for SQL. *Proceedings of the VLDB Endowment*, 4(1):22–33, Oct. 2010. doi: 10.14778/1880172.1880175 1, 2
- [13] D. Koop, C. Scheidegger, S. Callahan, J. Freire, and C. Silva. Viscomplete: Automating suggestions for visualization pipelines. *IEEE Transactions on Visualization and Computer Graphics*, 14(6):1691–1698, 2008. doi: 10.1109/TVCG.2008.174 2
- [14] B. Korvemaker and R. Greiner. Predicting unix command lines: Adjusting to user patterns. In *Proceedings of the Seventeenth National Conference on Artificial Intelligence and Twelfth Conference on Innovative Applications of Artificial Intelligence*, pp. 230–235, 2000. 2
- [15] M. Le Goc, L. H. Kim, A. Parsaei, J.-D. Fekete, P. Dragicevic, and S. Follmer. Zooids: Building blocks for swarm user interfaces. In *Proceedings of the 29th Annual Symposium on User Interface Software and Technology*, UIST '16, p. 97–109. ACM, New York, NY, USA, 2016. doi: 10.1145/2984511.2984547 1
- [16] J.-Y. Lo, D.-Y. Huang, T.-S. Kuo, C.-K. Sun, J. Gong, T. Seyed, X.-D. Yang, and B.-Y. Chen. Autofritz: Autocomplete for prototyping virtual breadboard circuits. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, CHI '19, p. 1–13. ACM, New York, NY, USA, 2019. doi: 10.1145/3290605.3300633 1, 2
- [17] J. Mackinlay. Automating the design of graphical presentations of relational information. *ACM Trans. Graph.*, 5(2):110–141, apr 1986. doi: 10.1145/22949.22950 1
- [18] J. Mackinlay, P. Hanrahan, and C. Stolte. Show me: Automatic presentation for visual analysis. *IEEE Transactions on Visualization and Computer Graphics*, 13(6):1137–1144, 2007. doi: 10.1109/TVCG.2007.70594 1
- [19] G. G. Méndez, U. Hinrichs, and M. A. Nacenta. Bottom-up vs. top-down: Trade-offs in efficiency, understanding, freedom and creativity with infovis tools. CHI '17, p. 841–852. ACM, New York, NY, USA, 2017. doi: 10.1145/3025453.3025942 2
- [20] G. G. Méndez, M. A. Nacenta, and U. Hinrichs. Considering agency and data granularity in the design of visualization tools. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, CHI '18, p. 1–14. ACM, New York, NY, USA, 2018. doi: 10.1145/3173574.3174212 2
- [21] G. G. Méndez, M. A. Nacenta, and S. Vandenheste. Ivolver: Interactive visual language for visualization extraction and reconstruction. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*, CHI '16, p. 4073–4085. ACM, New York, NY, USA, 2016. doi: 10.1145/2858036.2858435 2
- [22] B. Myers, S. E. Hudson, and R. Pausch. Past, present, and future of user interface software tools. *ACM Transactions on Computer-Human Interaction*, 7(1):3–28, Mar. 2000. doi: 10.1145/344949.344959 2
- [23] B. Saket, S. Huron, C. Perin, and A. Endert. Investigating direct manipulation of graphical encodings as a method for user interaction. *IEEE transactions on visualization and computer graphics*, 26(1):482–491, 2019. 2
- [24] B. Saket, H. Kim, E. T. Brown, and A. Endert. Visualization by demonstration: An interaction paradigm for visual data exploration. *IEEE transactions on visualization and computer graphics*, 23(1):331–340, 2016. 2, 4
- [25] A. Satyanarayan, R. Russell, J. Hoffswell, and J. Heer. Reactive vega: A streaming dataflow architecture for declarative interactive visualization. *IEEE Transactions on Visualization and Computer Graphics*, 22(1):659–668, 2016. doi: 10.1109/TVCG.2015.2467091 1
- [26] D. Schmandt-Besserat. *How writing came about*. University of Texas Press, 2010. 2
- [27] V. Setlur, E. Hoque, D. H. Kim, and A. X. Chang. Sneak Pique: Exploring Autocompletion as a Data Discovery Scaffold for Supporting Visual Analysis. In *Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology*, UIST '20, pp. 966–978. ACM, New York, NY, USA, Oct. 2020. doi: 10.1145/3379337.3415813 2
- [28] M. Tohidi, W. Buxton, R. Baecker, and A. Sellen. Getting the right design and the design right. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, CHI '06, p. 1243–1252. ACM, New York, NY, USA, 2006. doi: 10.1145/1124772.1124960 2
- [29] F. B. Viegas, M. Wattenberg, F. van Ham, J. Kriss, and M. McKeon. Manyeyes: a site for visualization at internet scale. *IEEE Transactions on Visualization and Computer Graphics*, 13(6):1121–1128, 2007. doi: 10.1109/TVCG.2007.70577 1
- [30] W. Willett and S. Huron. A constructive classroom exercise for teaching infovis. In *Pedagogy of Data Visualization Workshop at IEEE VIS 2016*, 2016. 2
- [31] K. Wongsuphasawat, D. Moritz, A. Anand, J. Mackinlay, B. Howe, and J. Heer. Voyager: Exploratory analysis via faceted browsing of visualization recommendations. *IEEE Transactions on Visualization and Computer Graphics*, 22(1):649–658, 2016. doi: 10.1109/TVCG.2015.2467191 2
- [32] T. Wun, J. Payne, S. Huron, and S. Carpendale. Comparing bar chart authoring with microsoft excel and tangible tiles. *Computer Graphics Forum*, 35(3):111–120, 2016. doi: 10.1111/cgf.12887 1, 2